02-614 Week 9
String Algorithms

Aidan Jan

October 29, 2026

RRR Vectors

An RRR vector is a data structure which stores bit vectors.

- The bit vector can be arbitrary, and store any type of data. (i.e., large numbers, sets, boolean array, etc.)

We will define the following:

- $\text{Rank}_1(i)$ = the number of 1's at or before position i in the bit vector.
- $\text{Select}_1(j)$ = the position of the j-th 1.

Access

Suppose we have the Rank function; how do we do access?

- Very easily. $S[i] = \text{Rank}_1(i) - \text{Rank}_1(i-1)$.
- This is because at a given position, the bit can either be a 0 or a 1. The rank changes if and only if it is a 1.

Goal

We want an efficient rank function since access needs to be fast. The naive method to making this work would be just preprocess the bit vector and store the rank indices. However, there is a second, much harder goal, which is to make it take less than n bits of storage.

The RRR Data Structure

Instead of storing the bit vector directly, we can divide the bit vector S into multiple blocks.

- Each blonk has a length of $u = O(\log n)$
- Let the index i refer to a block. Then,
  - w_i = the number of 1's in this block
  - P_i = a "pointer" to the block table
  - s_i = number of bits used to represent P_i
- Currently we don't understand the purpose of P_i and s_i, but they will be explained later.

Block Table

So what is a block table? It is similar to a hashmap, but for every possible block of size u. Each "bucket" in the block table represents a weight, or the number of 1's in the entire block. Each bucket contains two arrays, one storing the bit pattern, and the other storing the rank at every position.

For example, consider the block table for $u = 3$. A bit vector of length 3 has four buckets, one for each weight 0, 1, 2, 3.

```
w | (data, ranks)
0 | (000, 000)
--+---------------
1 | (001, 001)
  | (010, 010)
  | (100, 100)
--+---------------
2 | (011, 012)
  | (101, 112)
  | (110, 122)
--+---------------
3 | (111, 123)
```

A given block in s contains a P_i, which points to the corresponding pattern in the block table. This way, the actual bits of S don't have to be stored. Instead, we just store the pointer to the block table.

- The size of the pointer depends on how complex a given block is. Suppose our block of interest has a weight zero. In this case, our pointer doesn't need any bits if we know w_i, because if $w_i = 0$, then there is only one possible block.
- If instead in this case, $w_i = 1$, then we know it is in the $w = 1$ bucket. However, the pointer would have to be two bits in order to discern which pattern it is. Storing the size of the pointer is s_i, which is also stored in the block.

With this setup, we can now get the rank for any position in the *first* block. We can do this by looking up the pointer in the first block on the block table, and just use the rank. However, if we want to do this for other blocks, we need to know the number of 1's occurring before the block. In this case, we need to store a little bit more information in the blocks:

- f_i = sum of $w_j \; \forall j < i$. In other words, the number of 1's before block i.
- q_i = sum of $s_j \; \forall j < i$. This is the total length of pointers for all the blocks before the i-th block.

Now, we can define the Rank_1 function for all blocks:

$$\text{Rank}_1(i) = f_x + T[w_x][p_x][i - xu]$$

where $x = \lfloor \frac{i}{u} \rfloor$.

- f_x is the number of 1's before our block
- T is the block table
- $[w_x]$ is the weight bucket of the block table
- $[p_x]$ is the pattern in the weight bucket
- $[i - xu]$ is the local position in our current block

T can be built as a 3D matrix. The running time of rank is now constant time, since we only have to do array indexing.

Precompute

We have $O(1)$ rank, but how do we precompute everything? Additionally, how does this take less bits than n overall?

First, to precompute, we can just calculate all of the arrays of running totals and pointers, calculate the block table, then we are done. However, how do we compress it so it takes less than n bits?

- We can utilize a trick and delete the w_i array. It turns out that while it is needed for calculation, it does not need to be stored in the final structure. This is because the number of 1's present in block i can be found by subtracting $f_{i+1} - f_i$.
- It turns out we also don't need the s_i array, because that can be calculated in constant time by $q_{i+1} - q_i$.
- Deleting these two arrays already saves a lot of space, but does not get us all the way there.

Theorem: Tarjan + Yao, Pajh

Let $z_i, \ldots, z_k$ be integers, where

- $|z_i| = n^{O(1)}$ for some n. (i.e., integers aren't exponentially large)
- $|z_i - z_{i-1}| = O(\log n)$. (i.e., integers can't change too much between entries)

Then, the ordered list $z_1, \ldots, z_k$ can be represented with $O(K \log\log n)$ bits, with constant time access to any z_i.

Proof: Consider the array of length n integers. Let the array be divided into $O(\log n)$ blocks.

- Store the first integer, "keyframes" of each block as itself.
- Store all the other integers in the frame as deltas between the keyframe integer.

If we use this method for storage, then

- Keyframe integers take $O\left(\frac{k}{\log n} \cdot \log n\right) = O(k)$ bits.
- For the deltas, the integers for the deltas must be smaller than $\log(n)$, as that is a requirement for our list. However, that is a difference between two consecutive numbers, not the number and the keyframe. The maximum a number in a frame can deviate from its keyframe would be $O(\log n \cdot \log n) = O(\log^2 n)$. However, the number of bits we use to store that number is log of that. Therefore, we get $O(\log(\log^2 n)) = O(2 \log\log n)$ bits.

This form of compression gives us $O(k \log\log n)$ bits for all integers.

This theorem allows us to store the arrays of integers (namely, p_i, f_i, and q_i) in a $O(\log\log n)$ space. We just have to show that the numbers for each aren't too big and don't differ too much.

For $\hat{f}$ (or the vector containing all f_i):

- $f_i \leq n$ because at most we have n 1's in the entire array
- $|f_{i+1} - f_i$ because $u = O(\log n)$
- $k = \frac{n}{u} = \frac{n}{\log n}$. Therefore, we can store all f_i's in $O(\frac{n}{\log n} \log\log n)$.

This is impressive, since $O(\frac{n}{\log n} \log\log n)$ is much smaller than $O(n)$. In fact, as $n \to \infty$, this expression tends towards zero.

The same logic can hold for all of the q_i's.

Unfortunately, the same cannot be said for the p_i's. p_i points to a table with $\binom{u}{w_i}$ rows. (This number contains factorials.)

- Let $\mathcal{B}(w_i, u) := \lceil \log_2 \binom{u}{w_i} \rceil$. $\mathcal{B}$ is the amount of space to store all the p_i's.

Lemma: RRR data struct uses

$$O\left(\frac{n}{\log n}\log\log n + \sum_i \mathcal{B}(w, \log n) + \text{tables}\right)$$

How big are the tables?

Size of Tables

Tables have 2^u total cells, where u is the maximum number stored in a cell is u. This implies that a cell takes $\log u$ bits. Therefore, the total number of bits in the entire table is $u(\log u)2^u$ bits.

- Now, we use the fact that $u = O(\log n)$. In fact, we define it as $u = \epsilon \log n$, where $\epsilon < 1$.

Writing out the space complexity, we now have $(\epsilon \log n)\log(\epsilon \log n)2^{\epsilon \log n}$. This can simplify to $(\epsilon \log n)\log(\epsilon \log n)n^{\epsilon} \cdot 2^{\epsilon}$. Now, if we pick a small ϵ, say $\epsilon = \frac{1}{2}$, then

- The 2^{ϵ} term disappears because it is now a constant
- The overall space coplexity is now $O(\log n \log\log n\sqrt{n})$.
- The tables are now smaller than the f_i and q_i arrays, so they are no longer a significant part of the space complexity. They are included into the $O(\frac{n \log\log n}{\log n})$ term.

More Space Reductions

Now, we investigate the term

$$\sum_i^m \mathcal{B}\left(w_i, u\right)$$

First, we substitute the definition of $\mathcal{B}$.

$$\sum_{i=1}^{m} \left\lceil \log_2 \binom{u}{w_i} \right\rceil$$

Ceilings are annoying because they are discontinuous, but over the entire sum, they add 1. Therefore,

$$\begin{aligned} &\leq m + \sum \log_2 \binom{u}{w_i} \\ &= m + \log_2 \prod_{i=1}^{m} \binom{u}{w_i} \end{aligned}$$

Lemma. We know that:

$$\prod_{i=1}^{n} \binom{u}{w_i} \leq \binom{\sum_{i=1}^{m} u}{\sum_{i=1}^{m} w_i}$$

This is true because imagine a dartboard with i sections, each section containing u possible locations where darts could land. The original summation is like throwing w_i darts for each section i, the nCr function representing the number of combinations where the darts could land. The right side represents taking all the darts at the same time and throwing them into all buckets. It turns out the first is more constrained than the second, and therefore the second is greater.

The part that is interesting with this lemma is that the sum of all u_i's is just n, and the sum of all w_i's is the total count of 1's in the bit vector. We define that as w. Therefore,

$$\prod_{i=1}^{n} \binom{u}{w_i} \leq \binom{n}{w}$$

Therefore, we can substitute back:

$$m + \log_2 \prod_{i=1}^{m} \binom{u}{w_i}$$
$$\leq m + \log_2 \binom{n}{w}$$

We then notice the sum expression is very similar to the original definition of $\mathcal{B}$.

$$\leq m + \mathcal{B}(n, w)$$

The final space complexity we have is then

$$O\left(n\frac{\log\log n}{\log n} + \mathcal{B}(n, w)\right)$$

Wavelet Trees

Suppose $B = ACCGTTAG$.

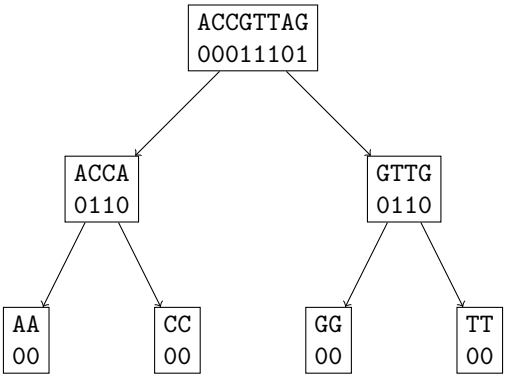
- $\text{rank}_c(i)$ = number of c's at or before position i
- $\text{select}_c(j)$ = position of the j-th c bit

What a wavelet tree does is split the alphabet used in the string into bit codes, and the represent the string using bit vectors.

- In our case here, the alphabet has four characters, so we can assign $A = 00, C = 01, G = 10, T = 11$

For representing the string B, the root of the tree is the first bit for each character. ($ACCGTTAG$ = 00011101) The two branches represents the characters beginning with a zero, and characters beginning with a one, respectively. So in this case, the left case would contain $ACCA$ = 0110, and the right case would contain $GTTG$ = 0110. The bits used in layer 1 use the 1-st (0-indexed) bits.

- We essentially represented our string as a bit vector..., but vertically.
- The bit vectors are actually stored as RRR vectors, which saves a lot of space.



Of course, we wouldn't store the characters at each layer, just the bits.